

12. Secure Communication using SSL/TLS Concepts: Understand and simulate secure data transmission using encryption protocols.

1. Practical Title

Secure Communication using SSL/TLS

2. Aim

To understand and simulate secure data transmission using **SSL/TLS (Secure Sockets Layer / Transport Layer Security)** protocols in Python using Google Colab.

3. Objectives

1. To understand the concept of SSL/TLS.
2. To create a secure SSL/TLS context using Python.
3. To establish a secure connection with an HTTPS server.
4. To identify the TLS version and cipher suite.
5. To demonstrate secure data transmission.
6. To verify data integrity using SHA-256 hashing.

4. Software Requirements

Requirement	Description
Platform	Google Colab
Programming Language	Python
Libraries	<code>ssl, socket, hashlib</code>
Internet	Required
Operating System	Any OS with a web browser

5. Theory

What is SSL?

SSL (Secure Sockets Layer) is a security protocol used to protect communication between a client and a server.

What is TLS?

TLS (Transport Layer Security) is the modern successor to SSL. It provides secure communication over networks such as the Internet.

For example:

HTTP → Unencrypted communication

HTTPS → HTTP + TLS security

When a user accesses a website using **HTTPS**, TLS helps protect the communication between the browser and the web server.

Main Security Services of TLS

1. Confidentiality

Data is encrypted so unauthorized people cannot easily read it.

2. Authentication

TLS certificates help verify the identity of the server.

3. Integrity

TLS helps detect whether data has been modified during transmission.

4. Secure Key Establishment

The TLS handshake establishes cryptographic parameters and keys for the secure connection.

6. Working of SSL/TLS

The basic communication process is:

Client

|

| TLS Connection Request

↓

Server



7. Algorithm

1. Start the Python program.
2. Import `ssl`, `socket`, and `hashlib`.
3. Create a secure SSL/TLS context.
4. Specify an HTTPS server and port 443.
5. Establish a normal TCP connection.
6. Wrap the TCP connection using SSL/TLS.
7. Display the TLS version and cipher suite.
8. Send an HTTPS request through the secure connection.
9. Receive the server response.
10. Generate a SHA-256 hash of sample data.
11. Compare the original and received hashes.
12. Display the result.
13. Stop the program.

8. Practical Implementation in Google Colab

Step 1: Open Google Colab

Open Google Colab and create a new notebook.

Create the following cells one by one.

Cell 1: Import Required Libraries

```
import ssl
```

```
import socket
```

```
import hashlib
```

```
print("Required libraries imported successfully.")
```

Output

Required libraries imported successfully.

Cell 2: Display SSL/OpenSSL Information

```
print("Python SSL Module Information")
```

```
print("-----")
```

```
print("OpenSSL Version:", ssl.OPENSSL_VERSION)
```

Example Output

Python SSL Module Information

OpenSSL Version: OpenSSL 3.x.x

The exact OpenSSL version may be different in Google Colab.

Cell 3: Create SSL/TLS Secure Context

```
context = ssl.create_default_context()
```

```
print("SSL/TLS secure context created successfully.")
```

```
print("Certificate Verification:", context.verify_mode)
print("Hostname Checking:", context.check_hostname)
```

Example Output

SSL/TLS secure context created successfully.

Certificate Verification: 2

Hostname Checking: True

Explanation

`create_default_context()` creates a secure configuration with certificate verification enabled by default.

Cell 4: Define Server Details

```
hostname = "www.google.com"
```

```
port = 443
```

```
print("Server:", hostname)
```

```
print("Port:", port)
```

```
print("Protocol: HTTPS")
```

Output

Server: www.google.com

Port: 443

Protocol: HTTPS

Explanation

HTTPS normally uses **port 443** for secure web communication.

Cell 5: Establish Secure TLS Connection

```
context = ssl.create_default_context()
```

try:

```
    with socket.create_connection((hostname, port)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as
secure_sock:

        print("Secure TLS connection established successfully!")
        print("TLS Version:", secure_sock.version())
        print("Cipher Suite:", secure_sock.cipher())
```

except Exception as e:

```
    print("Connection Error:", e)
```

Example Output

Secure TLS connection established successfully!

TLS Version: TLSv1.3

Cipher Suite: ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)

The exact TLS version and cipher may vary.

Cell 6: Display Server Certificate Information

```
context = ssl.create_default_context()
```

try:

```
    with socket.create_connection((hostname, port)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as
secure_sock:
```

```
certificate = secure_sock.getpeercert()

print("Server Certificate Information")
print("-----")

print("Subject:")
print(certificate.get("subject"))

print("\nIssuer:")
print(certificate.get("issuer"))

print("\nValid From:")
print(certificate.get("notBefore"))

print("\nValid Until:")
print(certificate.get("notAfter"))
```

```
except Exception as e:
```

```
    print("Error:", e)
```

Explanation

The server provides a digital certificate during the TLS connection. The client uses certificate verification to authenticate the server.

Cell 7: Send Data Through HTTPS

```
request = (
```

```

"GET / HTTP/1.1\r\n"
"Host: www.google.com\r\n"
"Connection: close\r\n\r\n"
)

context = ssl.create_default_context()

try:
    with socket.create_connection((hostname, port)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as
secure_sock:

            secure_sock.sendall(request.encode())

            response = secure_sock.recv(1024)

            print("Secure request sent successfully.")
            print("\nServer Response:")
            print(response.decode(errors="ignore")[:500])

except Exception as e:
    print("Error:", e)

```

Example Output

Secure request sent successfully.

Server Response:

HTTP/1.1 200 OK

...

The response may vary depending on the server.

Cell 8: Demonstrate Data Integrity

Now we will use **SHA-256** to demonstrate how data integrity can be checked.

```
message = "This is confidential data."
```

```
hash_value = hashlib.sha256(message.encode()).hexdigest()
```

```
print("Original Data:")
```

```
print(message)
```

```
print("\nSHA-256 Hash:")
```

```
print(hash_value)
```

Example Output

Original Data:

This is confidential data.

SHA-256 Hash:

<64-character hexadecimal value>

Cell 9: Verify Received Data

```
original_data = "This is confidential data."
```

```
received_data = "This is confidential data."
```

```
original_hash = hashlib.sha256(  
    original_data.encode()  
)hexdigest()
```

```
received_hash = hashlib.sha256(  
    received_data.encode()  
)hexdigest()
```

```
print("Original Hash:")  
print(original_hash)
```

```
print("\nReceived Hash:")  
print(received_hash)
```

```
if original_hash == received_hash:  
    print("\nData Integrity: VERIFIED")  
else:  
    print("\nData Integrity: FAILED")
```

Output

Data Integrity: VERIFIED

Cell 10: Demonstrate Modified Data

```
original_data = "This is confidential data."
```

```
received_data = "This is modified data."
```

```
original_hash = hashlib.sha256(  
    original_data.encode()  
)hexdigest()
```

```
received_hash = hashlib.sha256(  
    received_data.encode()  
)hexdigest()
```

```
print("Original Hash:")  
print(original_hash)
```

```
print("\nReceived Hash:")  
print(received_hash)
```

```
if original_hash == received_hash:  
    print("\nData Integrity: VERIFIED")  
else:  
    print("\nData Integrity: FAILED - Data was modified.")
```

Output

Data Integrity: FAILED - Data was modified.

This demonstrates that changing the data produces a different hash.

9. Complete Program

If your practical requires **one complete program**, use this:

```
import ssl
```

```
import socket
```

```
import hashlib
```

```
# -----
```

```
# Secure Communication using SSL/TLS
```

```
# -----
```

```
hostname = "www.google.com"
```

```
port = 443
```

```
print("SECURE COMMUNICATION USING SSL/TLS")
```

```
print("-----")
```

```
# Create secure SSL/TLS context
```

```
context = ssl.create_default_context()
```

```
print("\n1. SSL/TLS Context Created")
```

```
print("Certificate Verification:",
```

```
    context.verify_mode)
```

```
print("Hostname Checking:",
```

```
    context.check_hostname)
```

try:

```
# Create TCP connection
```

```
with socket.create_connection((hostname, port)) as sock:
```

```
# Establish TLS connection
```

```
with context.wrap_socket(
```

```
    sock,
```

```
    server_hostname=hostname
```

```
) as secure_sock:
```

```
    print("\n2. Secure TLS Connection Established")
```

```
    print("Server:", hostname)
```

```
    print("Port:", port)
```

```
# Display TLS information
```

```
print("TLS Version:",
```

```
    secure_sock.version())
```

```
print("Cipher Suite:",
```

```
    secure_sock.cipher())
```

```
# HTTPS request
```

```
request = (
```

```

"GET / HTTP/1.1\r\n"
"Host: www.google.com\r\n"
"Connection: close\r\n\r\n"
)

# Send request
secure_sock.sendall(request.encode())

# Receive response
response = secure_sock.recv(1024)

print("\n3. Secure Data Transmission Successful")
print("Server Response:")
print(response.decode(
    errors="ignore"
)[:300])

except Exception as e:

    print("\nConnection Error:", e)

# -----
# Data Integrity Demonstration

```

```
# -----

print("\n4. DATA INTEGRITY CHECK")
print("-----")

message = "This is confidential data."

hash_value = hashlib.sha256(
    message.encode()
).hexdigest()

print("Original Message:")
print(message)

print("\nSHA-256 Hash:")
print(hash_value)

print("\nData integrity mechanism demonstrated successfully.")
```

10. Expected Output

A typical output will look similar to:

SECURE COMMUNICATION USING SSL/TLS

1. SSL/TLS Context Created

Certificate Verification: 2

Hostname Checking: True

2. Secure TLS Connection Established

Server: www.google.com

Port: 443

TLS Version: TLSv1.3

Cipher Suite: ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)

3. Secure Data Transmission Successful

Server Response:

HTTP/1.1 200 OK

...

4. DATA INTEGRITY CHECK

Original Message:

This is confidential data.

SHA-256 Hash:

<64-character hexadecimal value>

Data integrity mechanism demonstrated successfully.

11. Observation Table

No.	Operation	Observation
1	SSL context creation	Secure context created

2	Server connection	HTTPS server connected
3	TLS handshake	Secure TLS session established
4	TLS version	TLS version displayed
5	Cipher suite	Encryption cipher displayed
6	HTTPS request	Data transmitted securely
7	SHA-256	Hash generated
8	Integrity check	Data modification detected

12. Advantages of SSL/TLS

1. Provides secure communication.
2. Protects confidential information.
3. Provides server authentication.
4. Helps maintain data integrity.
5. Protects information during network transmission.
6. Used by HTTPS websites.

13. Applications

SSL/TLS is commonly used in:

- Online banking
- E-commerce websites
- Email communication
- Online payments
- Login pages
- Cloud applications
- APIs
- HTTPS websites
- Secure web applications

14. Precautions

1. Use a trusted TLS certificate.
2. Enable certificate verification.
3. Verify the server hostname.

4. Avoid disabling TLS security checks.
5. Use modern TLS versions.
6. Do not send sensitive information to unknown servers during testing.